

*M. Sc. in Computer, Communication, and Information Sciences
Major in Communications Engineering*

Master's Thesis

Evolving Trends in Software Performance and Energy Consumption: An Analysis of Compilers and Interpreters over Time

M. Sc. Student: César Iván Olvera Espinosa

Supervisor & Advisor: Jukka Manner

Date: 24 February 2024

1. Introduction
2. Programming Language
 - a. Compilers and Interpreters
3. Case Study: Investigating compiler and interpreter trends
 - a. Methodology
 - b. Results and Observations
 - a. Python
 - b. C++
 - c. JavaScript
 - d. Java
 - c. Discussion
4. Conclusion and future work

1. Introduction

🧐 Motivation

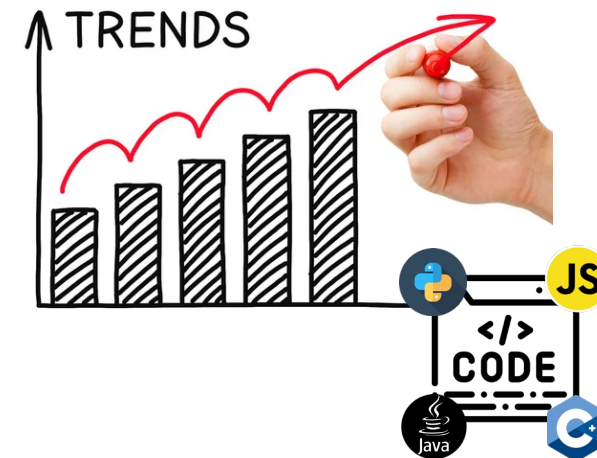
- Concern about the energy consumption of application software due to the impact of climate change
- Software applications lead the computer system (hardware) in executing specific tasks which require considerable hardware resources to execute them affecting directly to the energy consumption
- The demand for software applications and devices has increased significantly in the last few years

🎯 Problem statement

- Understand the root causes of the decreasing software performance observed nowadays and identify how impact energy consumption
- Observe the trends and changes over time in the most common programming languages.
- Identify the released versions that favourably impact energy consumption and performance.

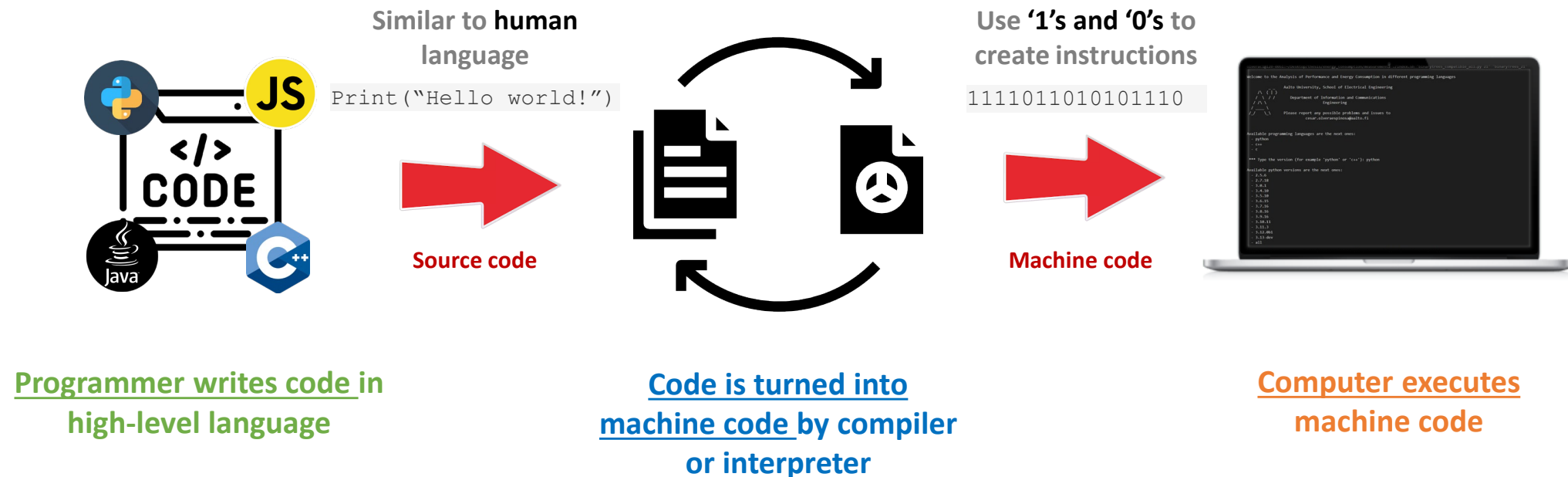
? Research questions:

1. How is the trend of compilers and interpreter in the most common programming languages in terms of energy consumption and performance over the time?
2. How is the impact of selecting a specific compiler or interpreter version in the most common programming languages in terms of energy consumption and performance?



2. Programming languages

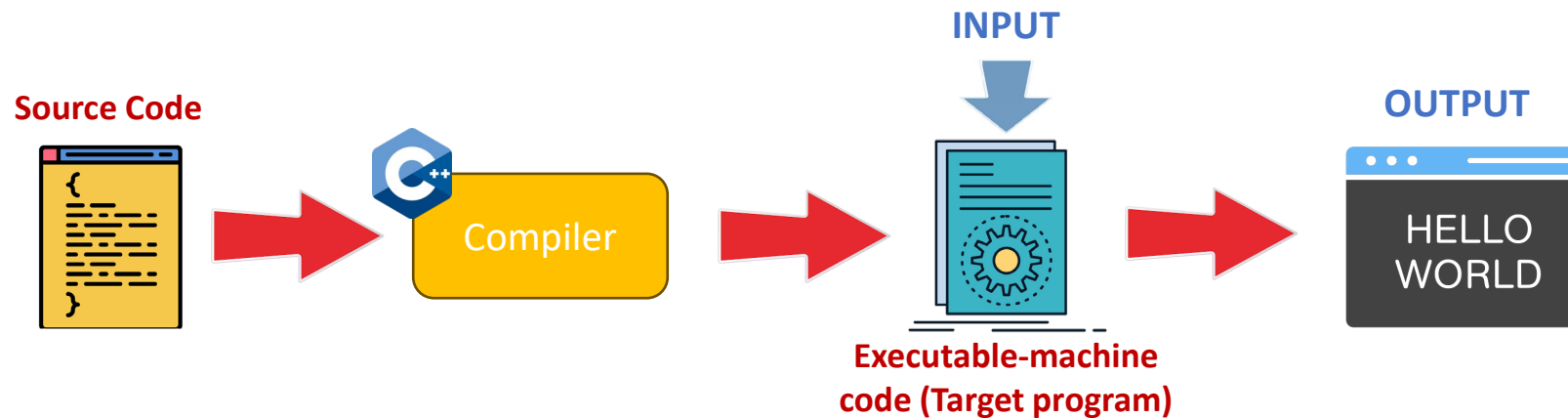
- 💡 Computers only understand a language of zeros and ones called *machine language (low-level language)*, and programming languages are crucial to communicate with the computer through resembled natural languages such as English to appropriately execute tasks [1] known as *high-level languages*.



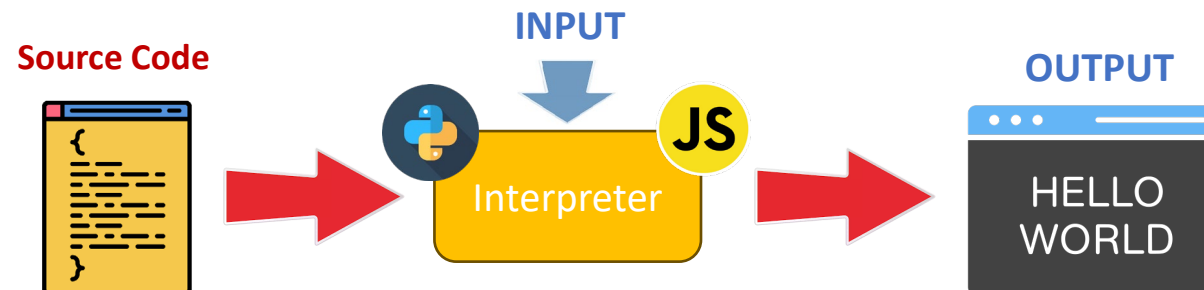
2. Programming languages

a) Compilers and Interpreters

Compiler: Convert the entire source code into machine code and then it can be called by the user. In addition, it reports any errors in the source code that it detects during the translation process and usually is much faster than an interpreter



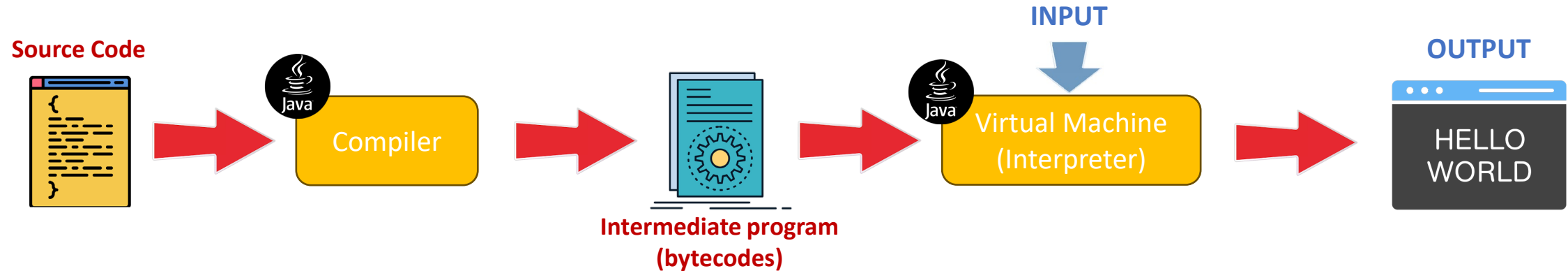
Interpreter: Directly execute the operations specified in the source program statement by statement. It can give better error diagnosis.



2. Programming languages

a) Compilers and Interpreters

Hybrid compiler or Semi-compiler: Combine both compilation and interpretation. The main advantage is that bytecodes can be interpreted on another machine.



4. Case Study: Investigating compiler and interpreter trends

c) Methodology

 Programming Languages:

	Programming Language	Interpreter/Compiler	The oldest version	The newest Version	Years
	Python	Cpython (interpreter)	2.5.6	3.11.3, 3.12 (beta) and 3.13 (alpha)	2011 – 2023 (12 years)
	C++	G++ (compiler)	4.4	13	2012 – 2023 (11 years)
	Java	Javac (JIT-compiler)	8 (1.8.0_382)	20.0.2	2014* – 2023 (9 years)
	JavaScript	Node (interpreter)	0.8.28	20.5.1	2014 – 2023 (9 years)

** First release of Java 8 due to this version is supported up to December 2030 by Oracle*

4. Case Study: Investigating compiler and interpreter trends

c) Methodology

Programs:

- Computing tasks by CLBG (Computer Language Benchmark Game) by selecting the fastest codes:
 - N-body (CPU usage through mathematical calculations)
 - Binary-Trees (Memory usage through increasing exponentially with the depth of the binary trees)

Set-up:

- Laptop Dell Inc. Latitude 7400 (Ubuntu 22.04.2 LTS 64-bit)
 - Memory 16 GB
 - Intel Core i5-8365U CPU @ 1.60 GHz x 8
 - Disk Capacity 512.1 GB

4. Case Study: Investigating compiler and interpreter trends

c) Methodology

Measurement software

- Turbostat using Intel RAPL (energy measurement)
- Perf (performance measurement)

Parameters:

- Energy Consumption of Package Processor (includes the cores and integrated graphics)
- Energy Consumption of RAM
- Time elapsed

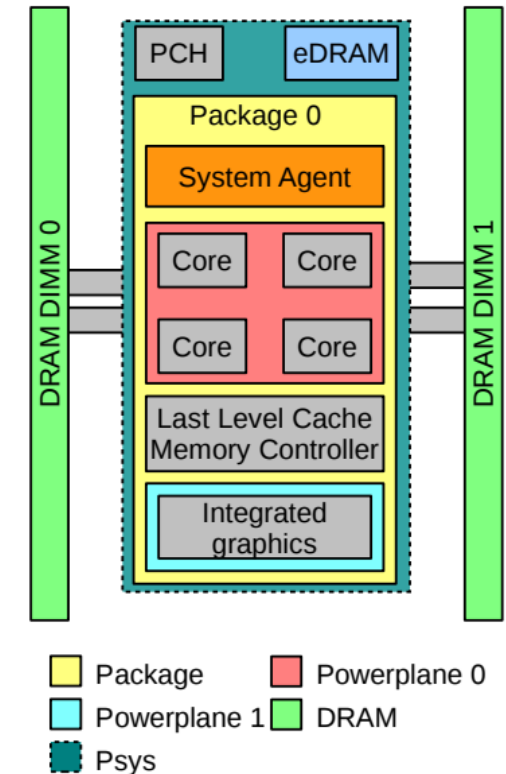


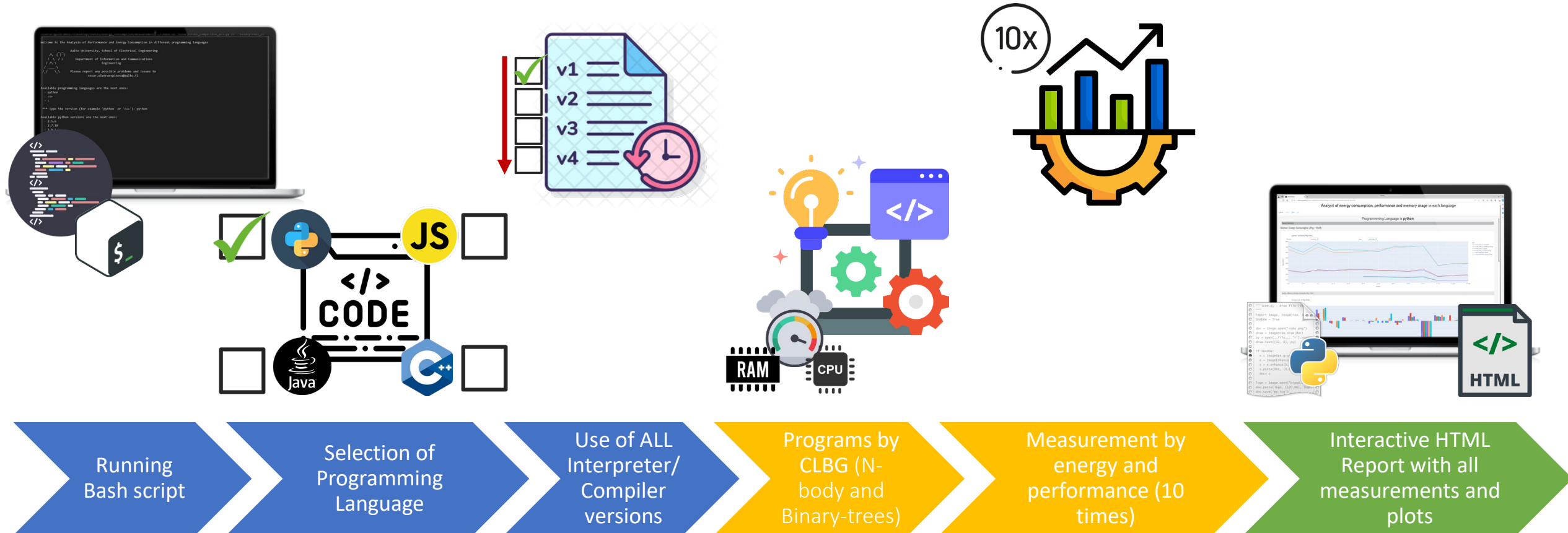
Fig. 1. Power domains supported by RAPL.

4. Case Study: Investigating compiler and interpreter trends

BASH Script

Running programming language selected

Interactive HTML report



Case Study: Investigating compiler and interpreter trends

RESULTS AND OBSERVATIONS

*All programs and versions of N-body and Binary-Trees in the programming languages
(Python, C++, Java, JavaScript)*

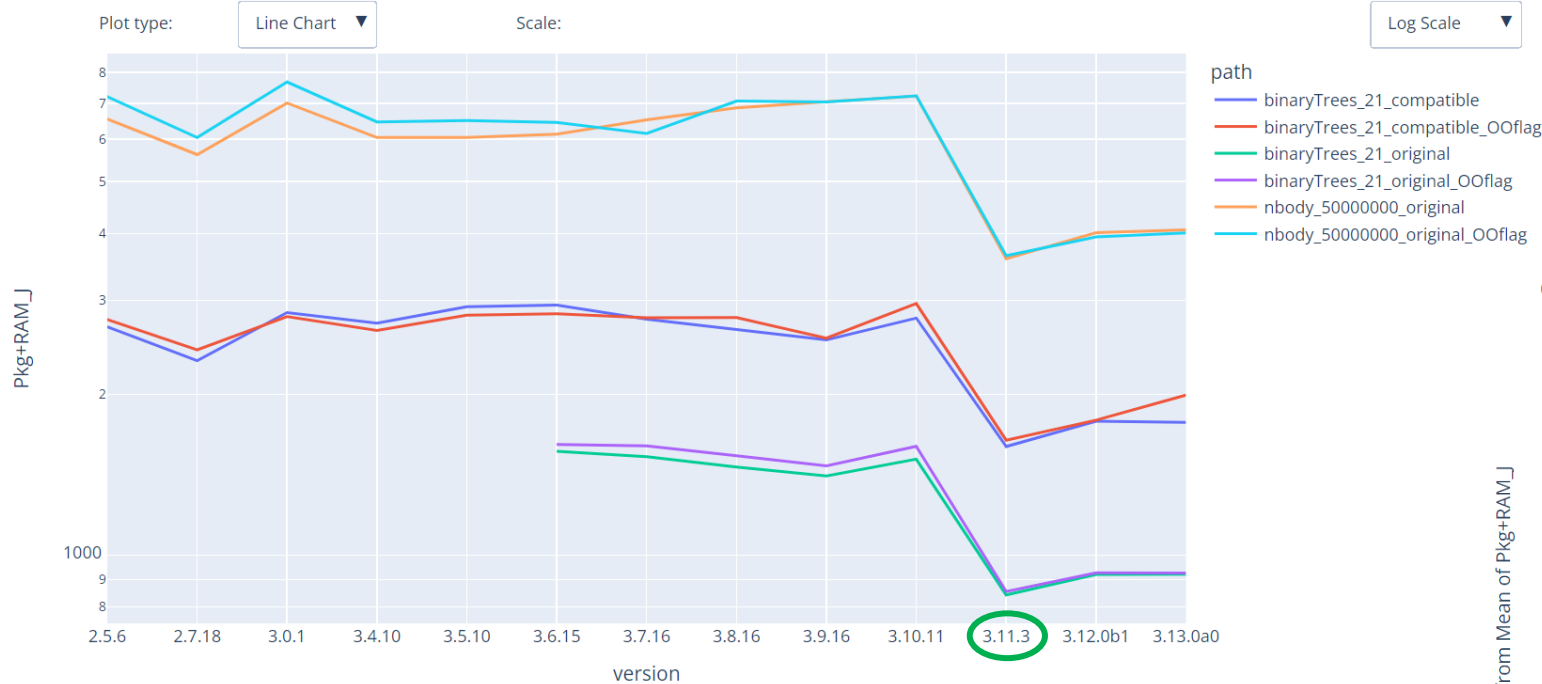
4. Case Study: Investigating compiler and interpreter trends



Python

python - turbostat_Pkg+RAM_

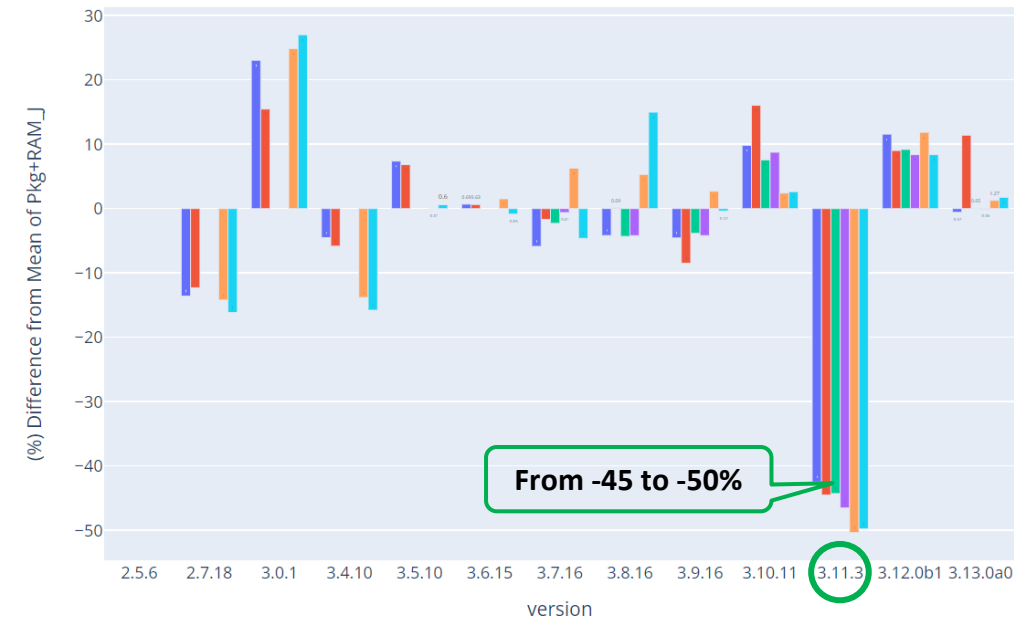
Energy Consumption



The general trend indicates that there has been **a decrease of energy consumption and time elapsed in the last three versions.**

About **45-50% decreasing** of energy consumption and time elapsed in all programs executed **from the version 3.11.3**

Comparison of Pkg+RAM_



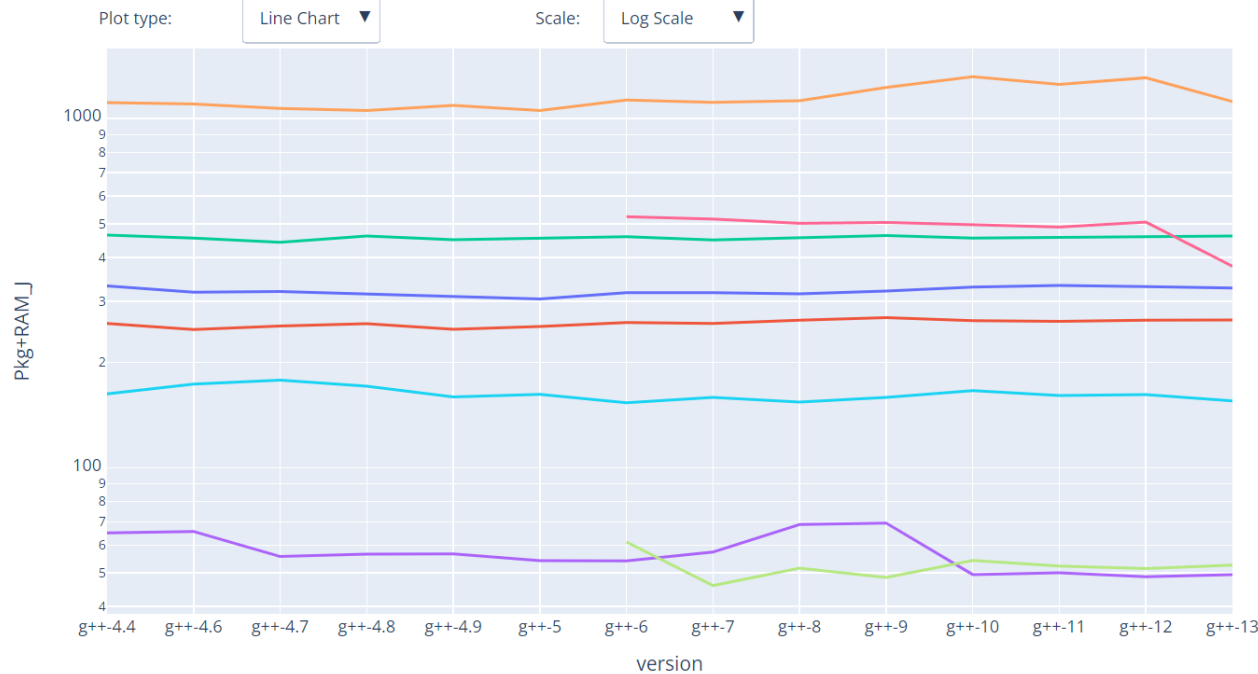
4. Case Study: Investigating compiler and interpreter trends



C++

c++ - turbostat_Pkg+RAM_J

Energy Consumption



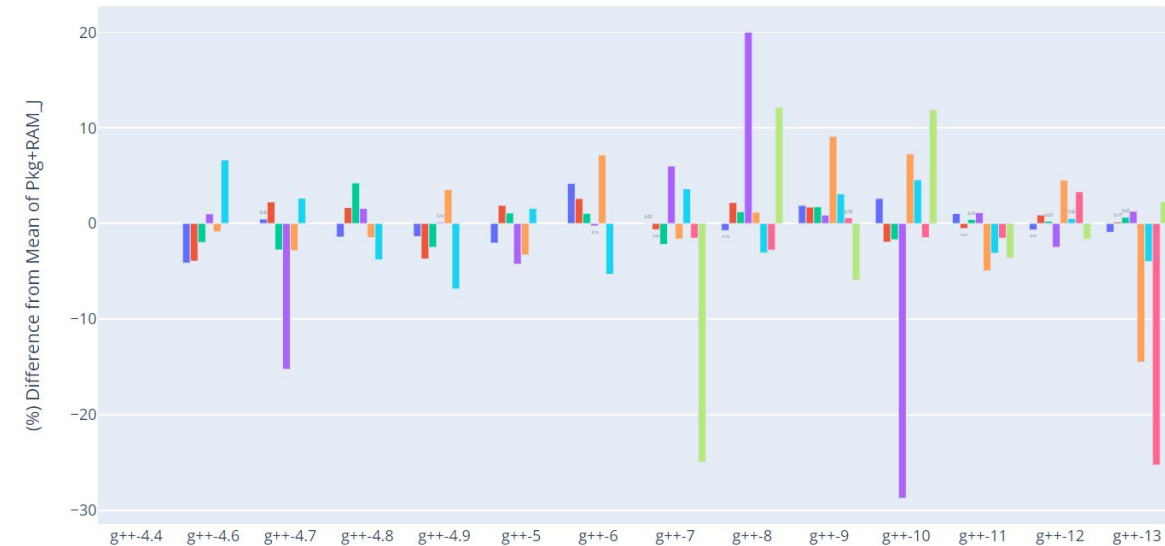
- The best cases for both Binary-trees and N-body has been more affected in some released versions using O3 flag

path

- binaryTrees_v2_21_original
- binaryTrees_v2_21_original_O3flag
- binaryTrees_v6_21_original
- binaryTrees_v6_21_original_O3flag
- nbody_50000000_v2_compatible
- nbody_50000000_v2_compatible_O3flag
- nbody_50000000_v2_original
- nbody_50000000_v2_original_O3flag

The general trend indicates that it **has not varied much, except in few cases where decreases around 10-20%**. The energy consumption trend is **more constant with a variation about 5% in the last three versions**.

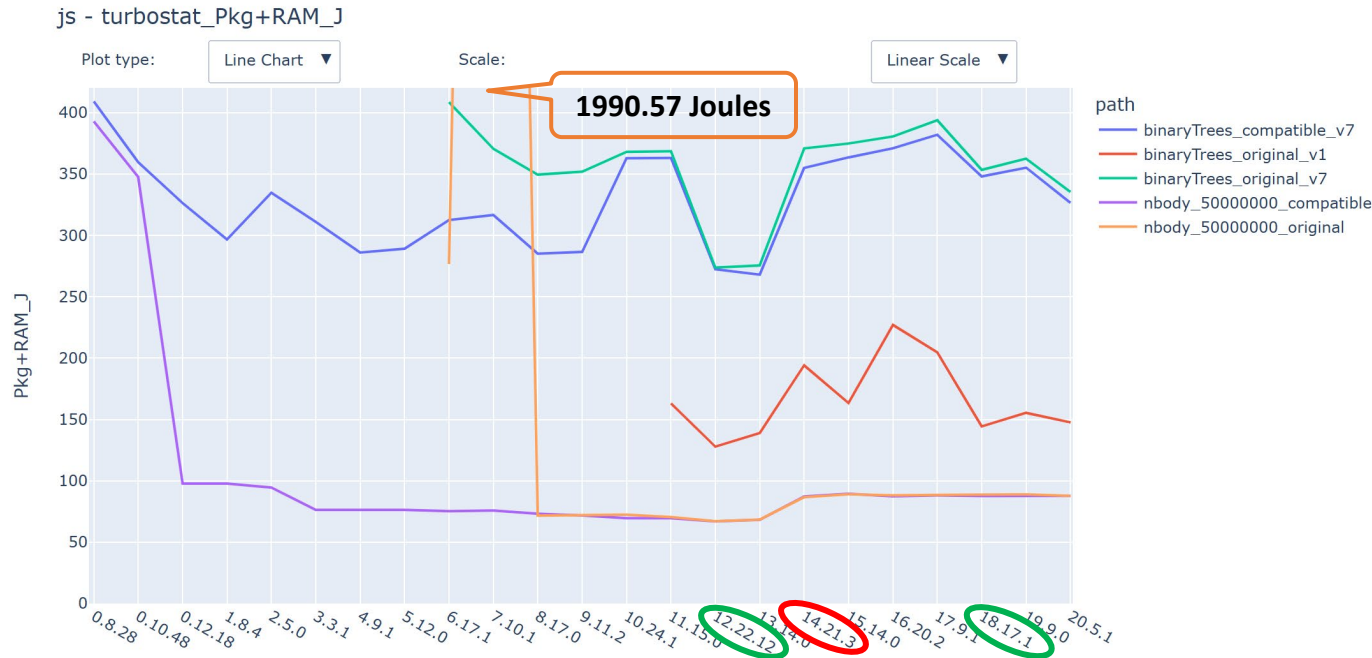
Comparison of Pkg+RAM_J



4. Case Study: Investigating compiler and interpreter trends

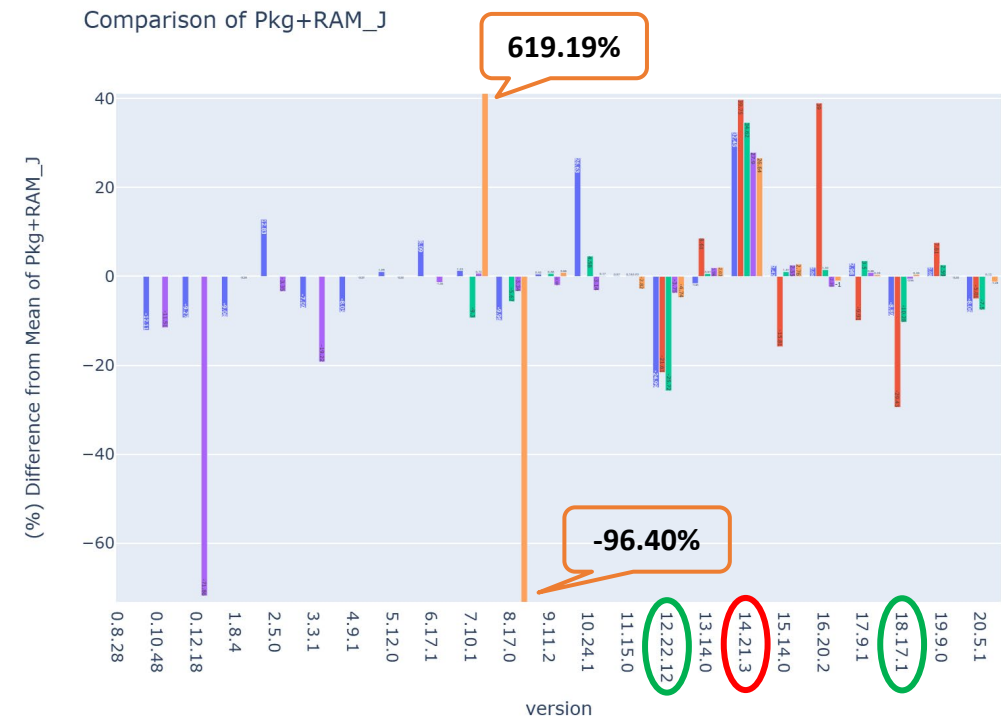
JS JavaScript

Energy Consumption



- Only the N-body case (original version code) is the worst scenario:
 - From version 6.17.1 to 7.10.1, the energy consumption trend **increases 619.19%**.
 - From version 7.10.1 to 8.17.0, the energy consumption trend **reduces 96.40%**
- The energy consumption trend has impacts through the released versions for all cases:
 - Version 14.21.3 increases **around 26-40%**
 - Version 12.22.12 decreases **around 20-25%**
 - Version 18.17.1 decreases **around 10-30%**

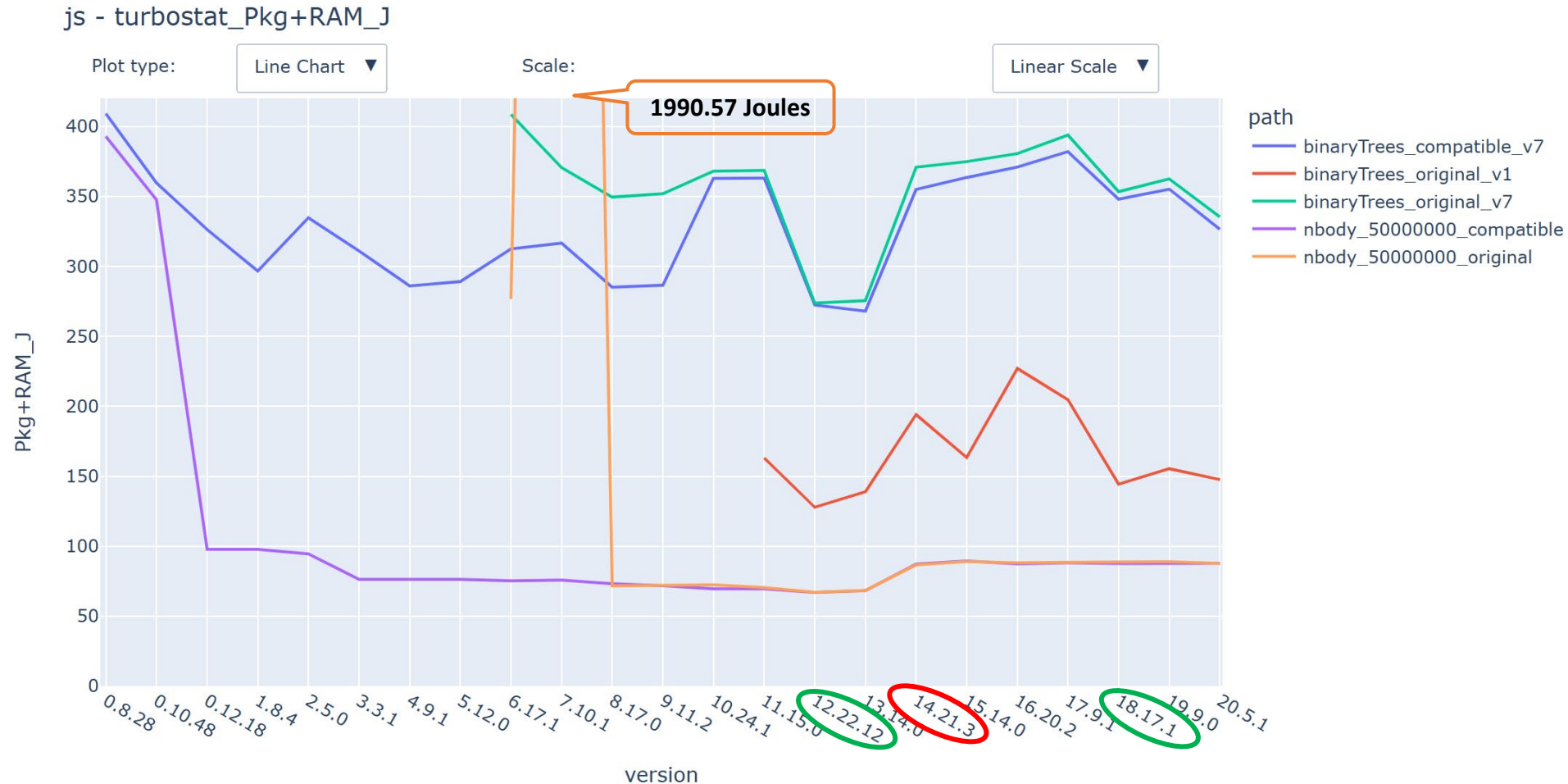
The general trend indicates that the last three versions are more stable around 7-8% (18.17.1, 19.9.0, 20.5.1)



4. Case Study: Investigating compiler and interpreter trends

JS JavaScript

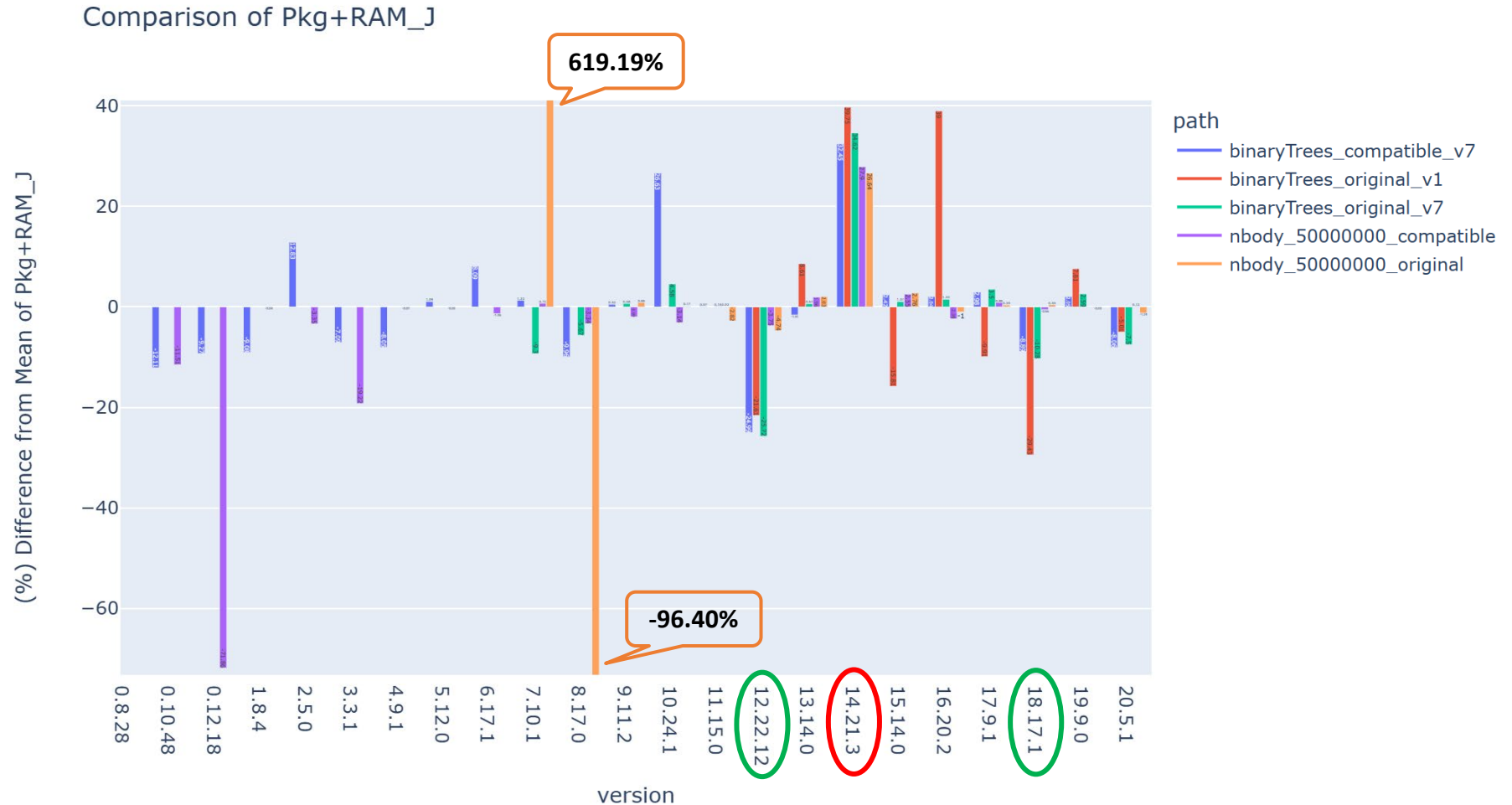
Energy Consumption



4. Case Study: Investigating compiler and interpreter trends



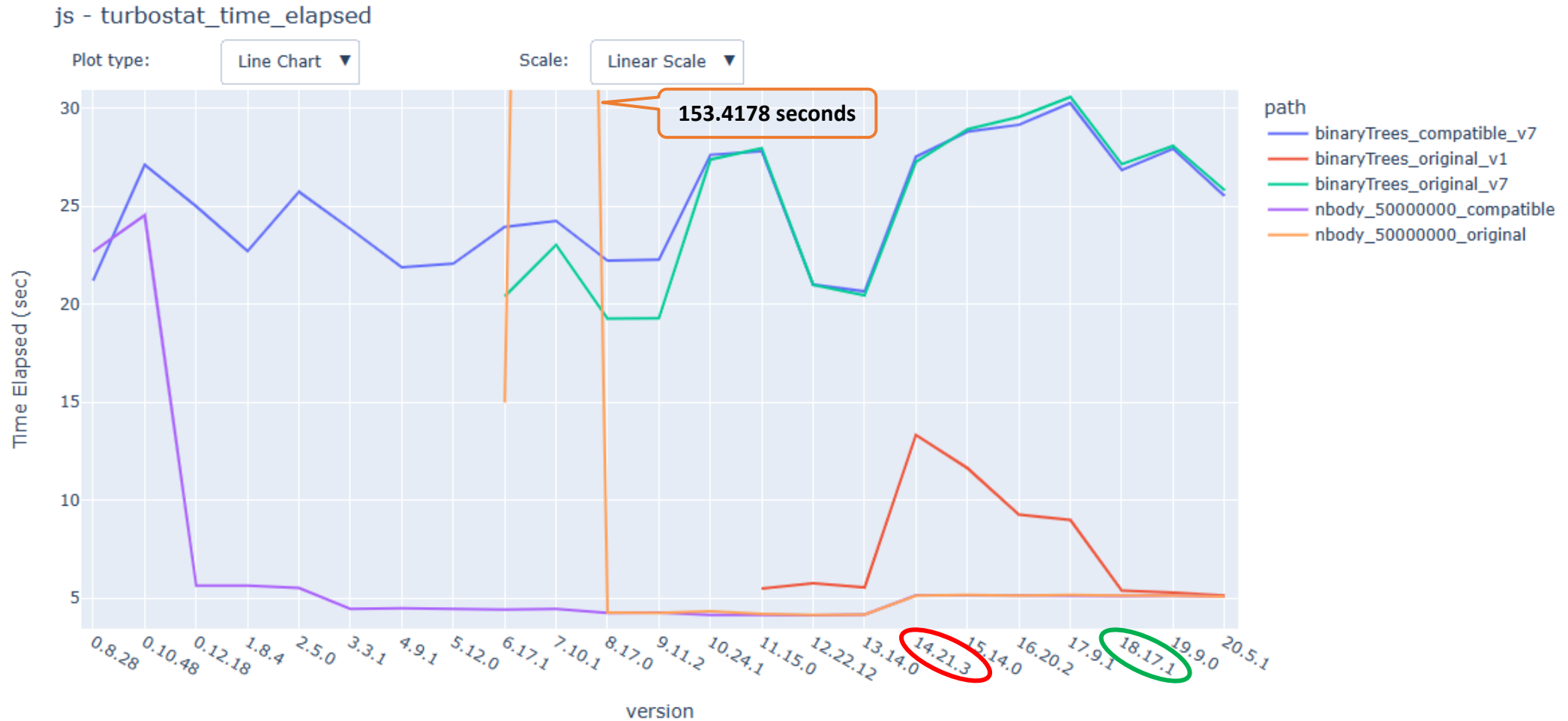
Energy Consumption



4. Case Study: Investigating compiler and interpreter trends

JS JavaScript

Energy Consumption



4. Case Study: Investigating compiler and interpreter trends

JS JavaScript

Energy Consumption



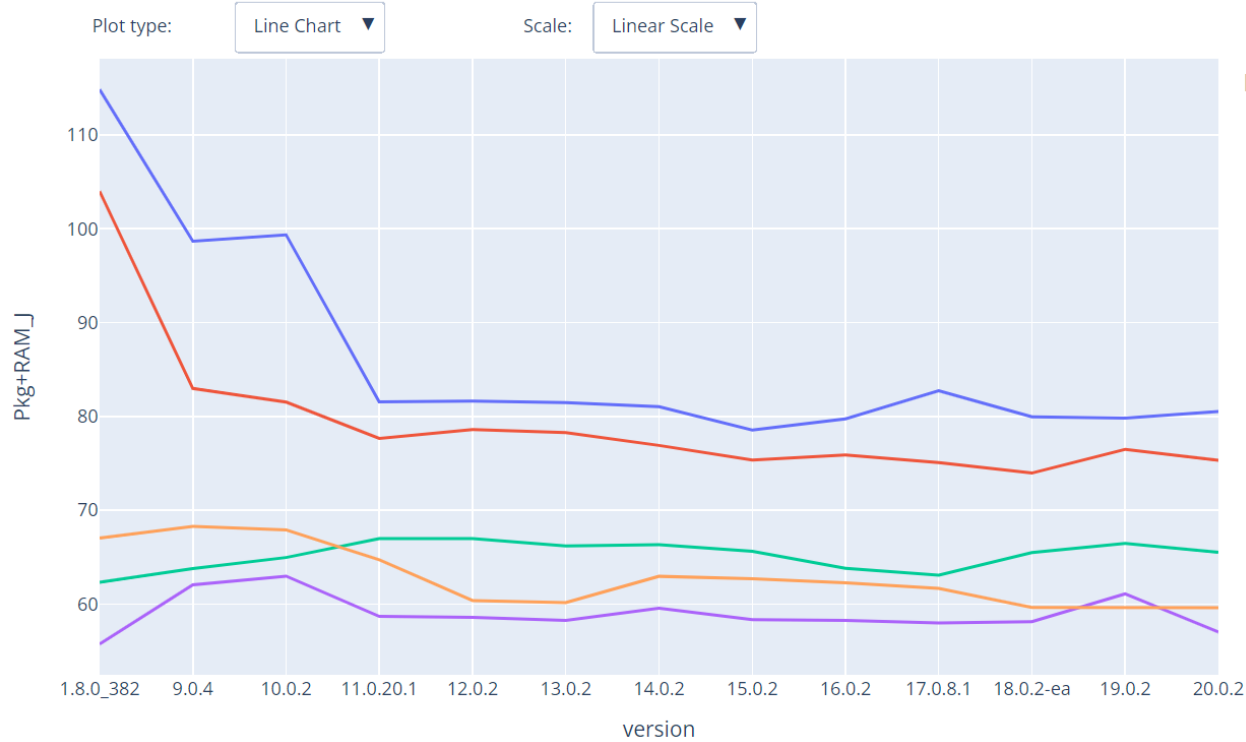
4. Case Study: Investigating compiler and interpreter trends



Java

Energy Consumption

java - turbostat_Pkg+RAM_J

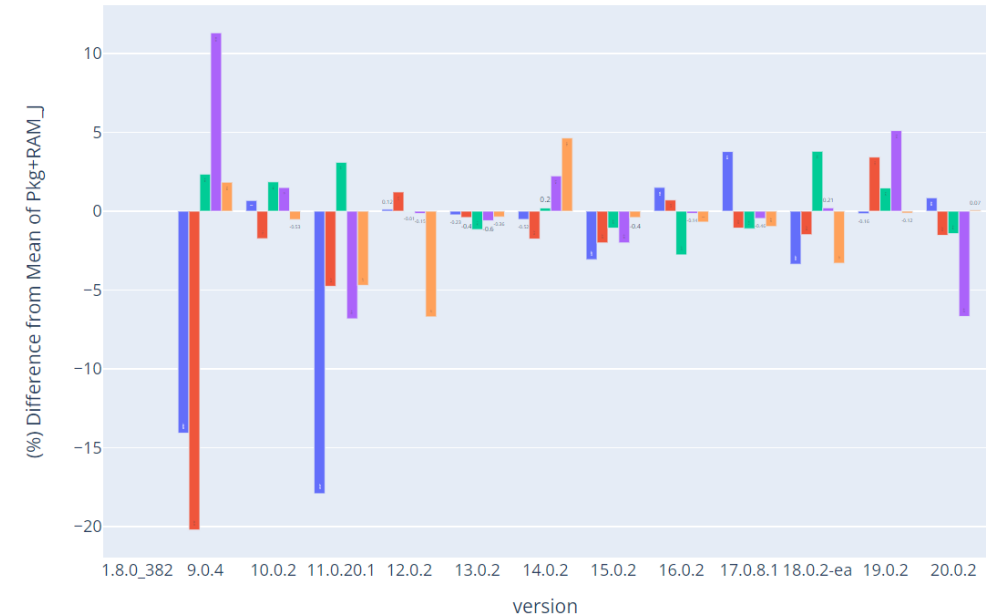


💡 The energy consumption and time elapsed trend starts to be **more constant from the version 11.0.20.1 (varying about 5%)**

Observations:

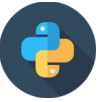
- 11.0.20.1 reduced the energy consumption in most of the cases (around 5-17%)

Comparison of Pkg+RAM_J

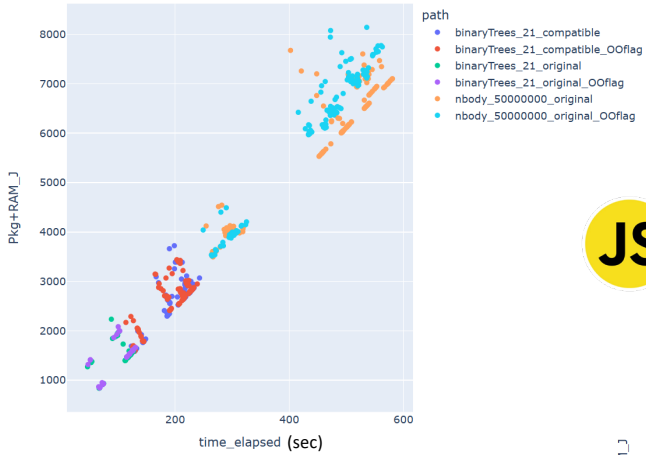


4. Case Study: Investigating compiler and interpreter trends

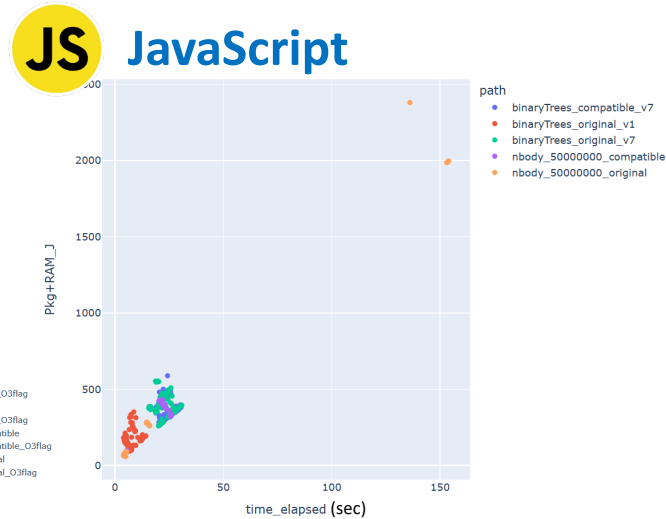
Energy Consumption and Time Elapsed



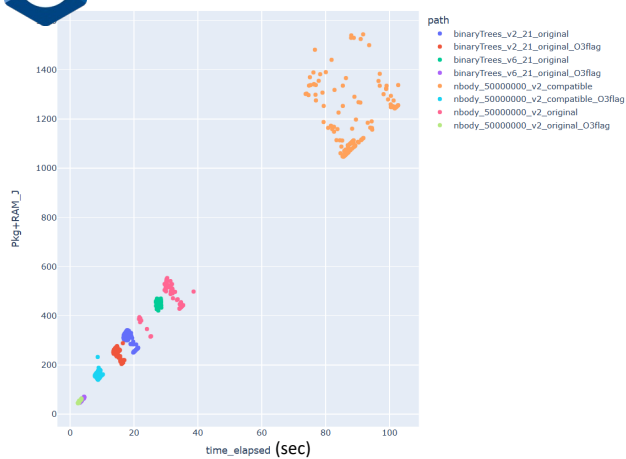
Python



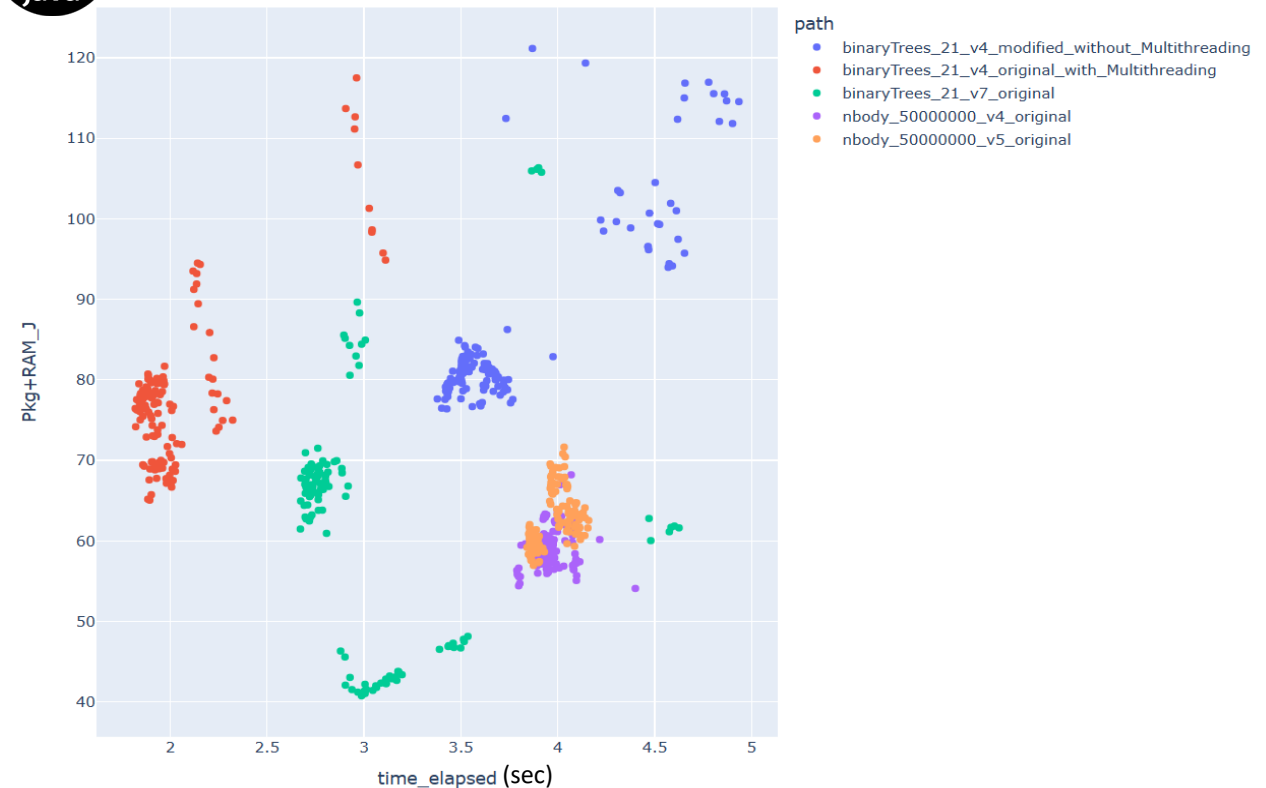
JavaScript



C++



Java



💡 In Java, there is a **low correlation** between the energy consumption and time elapsed

4. Case Study: Investigating compiler and interpreter trends

b) Results and Observations



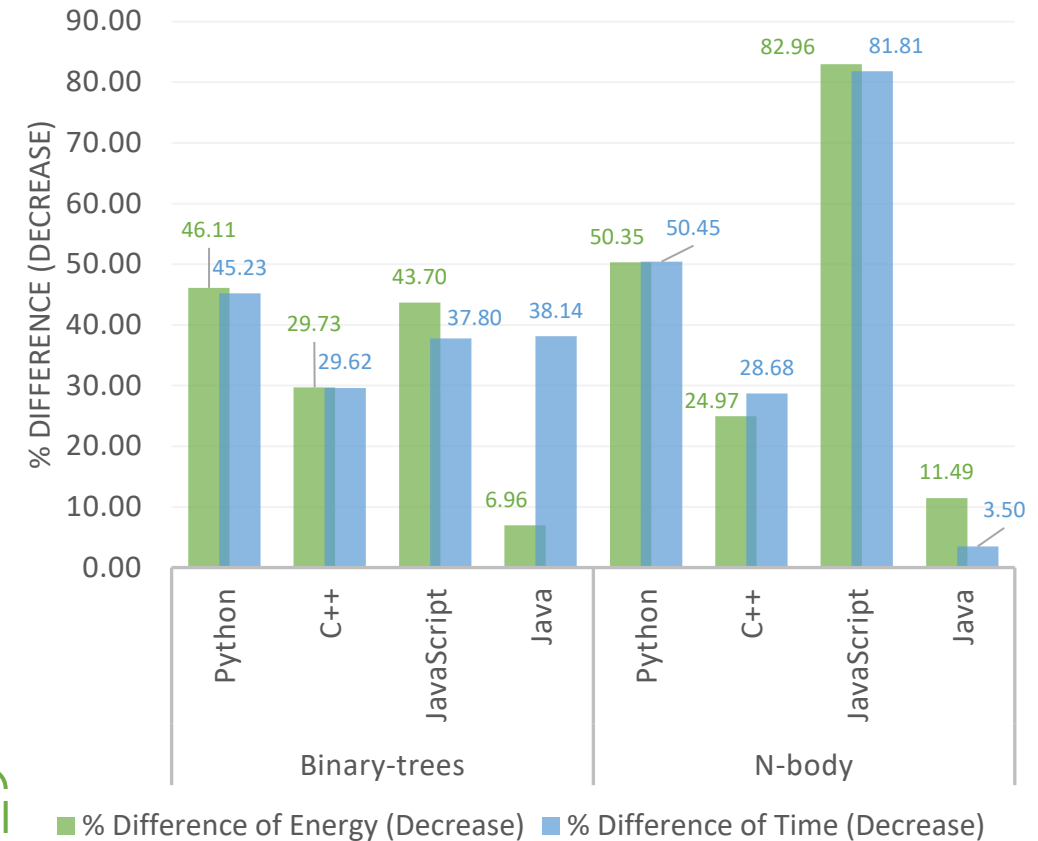
Comparing **the BEST and WORST interpreter/compiler versions used for the BEST scenario in each programming language for both Binary-trees and N-body.**

The next parameters present the best scenarios:

	Python	C++	JavaScript	Java
The BEST Interpreter/Compiler Version	3.11.3	g++12 (Binary-trees) g++7 (N-body)	12.22.12	1.8.0_382
The WORST Interpreter/Compiler Version	3.6.15 (Binary-trees) 3.10.11 (N-body)	g++9 (Binary-trees) g++6 (N-body)	16.20.2 (Binary-trees) 0.8.28 (N-body)	16.0.2 (Binary-trees) 10.0.2 (N-body)
Difference of energy (Joules)	721.23 (Binary-trees) 3637.24 (N-body)	20.62 (Binary-trees) 15.29 (N-body)	99.29 (Binary-trees) 325.98 (N-body)	4.67 (Binary-trees) 7.24 (N-body)
Difference of time elapsed (secs)	55.51 (Binary-trees) 271.22 (N-body)	1.30 (Binary-trees) 1.01 (N-body)	3.5 (Binary-trees) 18.56 (N-body)	1.71 (Binary-trees) 0.14 (N-body)

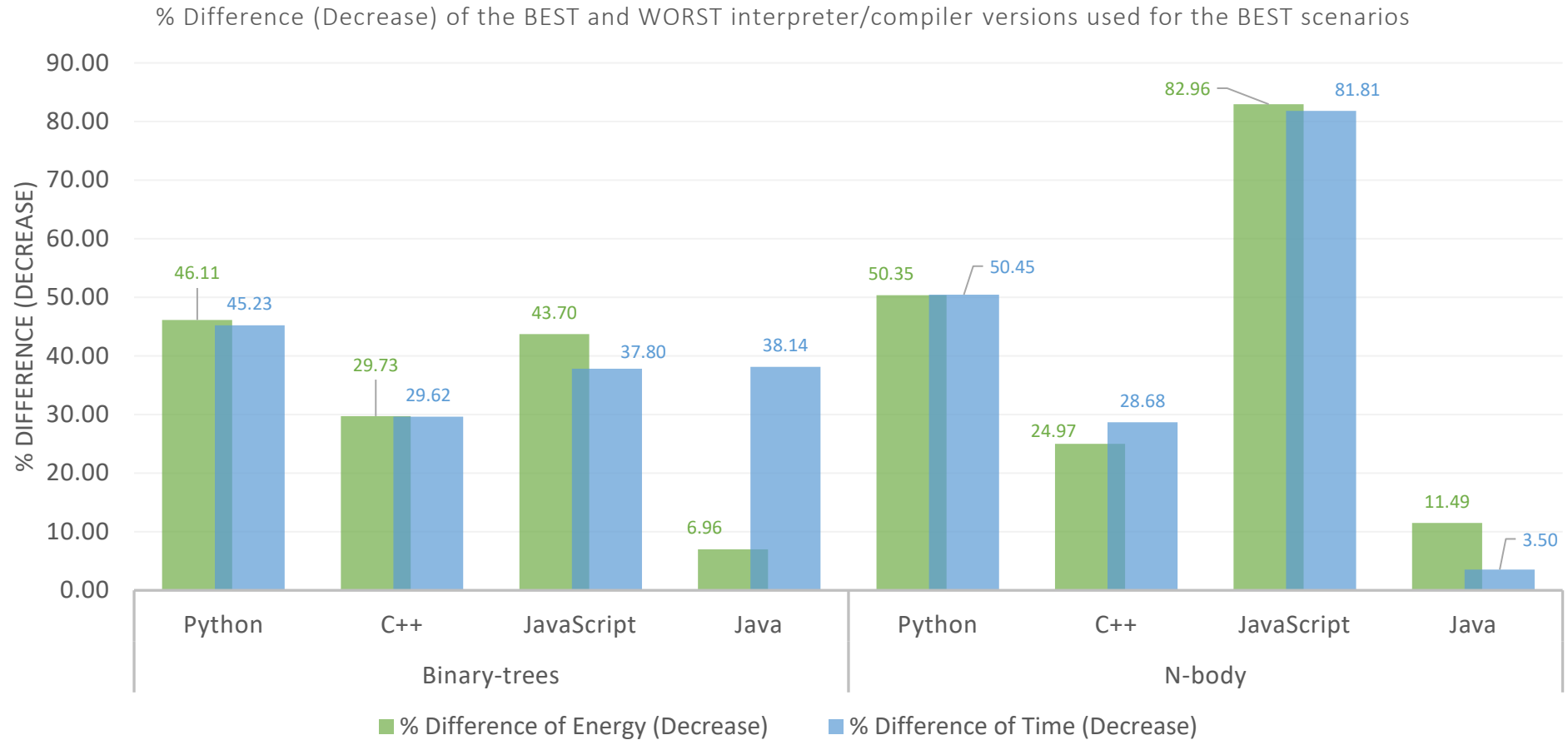
💡 **Python and JavaScript has more impact** in the energy consumption and time elapsed based on the selection appropriately of interpreter version

% Difference (Decrease) of the BEST and WORST interpreter/compiler versions used for the BEST scenarios



4. Case Study: Investigating compiler and interpreter trends

b) Results and Observations

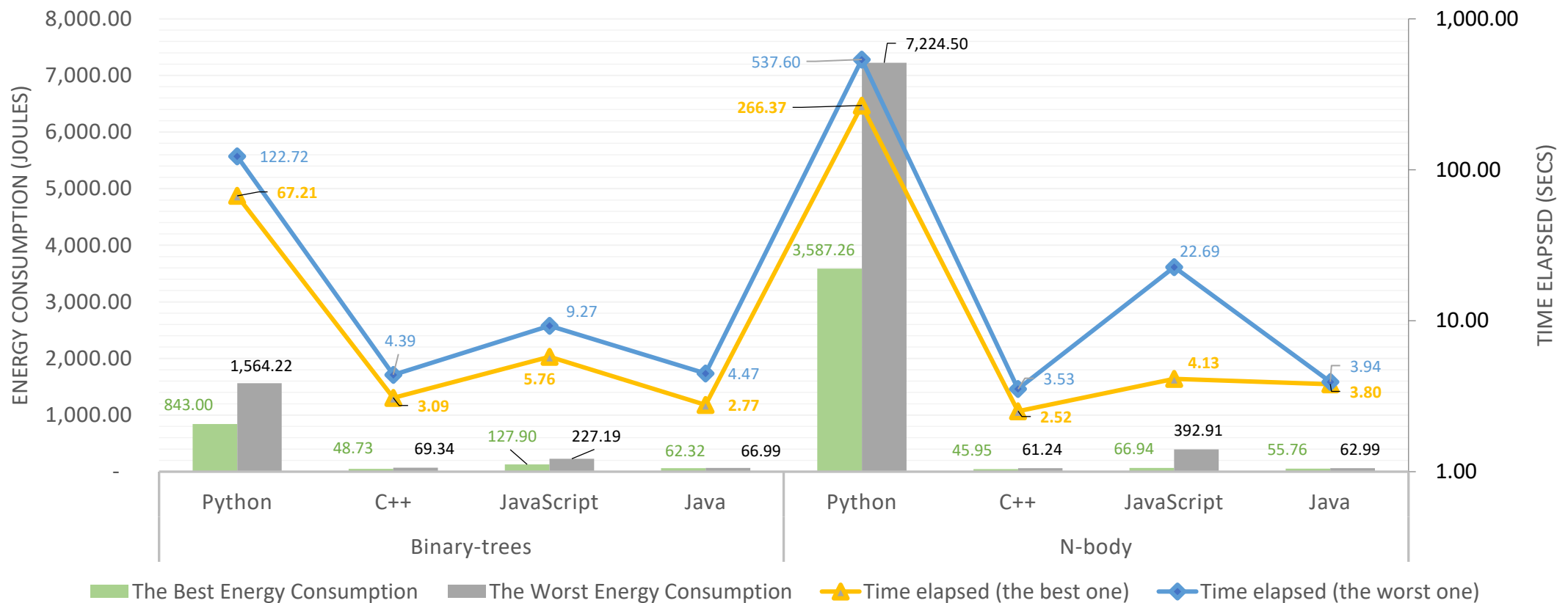


4. Case Study: Investigating compiler and interpreter trends

b) Results and Observations



Energy Consumption and Time Elapsed of the BEST scenario selecting the BEST and WORST interpreter/compiler versions



4. Case Study: Investigating compiler and interpreter trends

a) Discussion

	Python	C++	JavaScript	Java
General trend	Decreases (from version 3.11.3 about 45-50%)	More constant about 5% (from the last three versions)	Varies over time, but reduces about 10-30% and then constant about 7-8% (from the version 18.17.1)	More stable with a variation about 5% (from version 11.0.20.1 reducing about 5-17%)
The BEST version(s)	From version 3.11.3 (decrease about 45-50%)	-	Version 12.22.12 (decrease about 32-96%)	Version 1.8.0_382 (decrease about 6-28%)
The WORST version(s)	3.0.1 and 3.10.11 (increase about 80-111%)	-	6.17.1 and 7.10.1 (increase about 49-2867%) 14.21.3 (increase about 26-40%)	-
Original vs Compatible Code	Original (decrease around 47-48%)	Original (decrease around 66-95% using BOOST libraries)	Compatible (decrease about 14-96% using 'var' instead of 'const' or 'let')	-
Version code (by CLBG)	-	Decrease about 28-81% according to the code	Decrease about 49-55% according to the code	Decrease about 13-40% according to the code
Extra parameters	OO flag (reduces about 1-3%)	O3 flag (decrease about 19-90%)	-	-
Correlation (time elapsed and energy consumption)	High	High	High	Low

6. Conclusion

- Important software engineering factors to consider:
 1. **Selection of an appropriate version of compiler/interpreter** can have an impact especially in Python and JavaScript while Java does not have a high impact.
 2. **Code with full compatibility** (old and new versions) can have an impact in the energy consumption (except Java due to it did not present any problem)
 3. **Optimization flags has a high impact** reducing the energy consumption and time elapsed in C++.
- The energy consumption and time elapsed trends:
 - Java and C++ have more stable and constant performance
 - JavaScript varies over time, but the last three versions are more constant.
 - Python is decreasing the energy consumption and time elapsed from the last three versions
- Most of the measurements the energy consumption has a high correlation with the time elapsed, except Java in some cases.
- There are other performance parameters which are correlated to the energy consumption and time elapsed that can be analysed and observed to develop and improve a code

7. Future work

- Applied the same analysis for more programs to observe if there are some changes in the trend.
- Verify the trend of other common programming languages such as C, PHP, etc.
- There are more interpreters and compilers options for those programming languages that can be analyzed to see if they provide a better performance and energy consumption. For instance, Python has Condo or Pypy or C++ has Clang as well.

Thanks!
Kiitos!
¡Gracias!



aalto.fi



Aalto-yliopisto
Aalto-universitetet
Aalto University